

How do I authenticate OAuth and http2?

Asked 1 year ago Modified 1 year ago Viewed 1k times



Sign in to Stack Exchange with Google



Tony Zoppi

ajzopp@gmail.com

Continue as Tony

To create your account, Google will share your name, email address, and profile picture with Stack Exchange. See Stack Exchange's [privacy policy](#) and [terms of service](#).

Problem Description

3

I am trying to implement OAuth 2.0 for a web application which is [documented here](#). I believe I am using the "authorization code" and then use that code to get my request for the access token fails as a "HTTP 400 Bad Request" each time I attempt it.

I am seeking help to find my error. Thank you for any help offered.

Security for Encrypted Data

After running this script in RMarkdown and using reprex, I manually replaced all potentially sensitive data with "ENCRYPTEDSTRING{1-6}". Aside from that, this markdown file shows exactly what I got when run.

Packages Used



```
library(httr2)
library(openssl)
#> Linking to: OpenSSL 3.3.0 9 Apr 2024
library(tidyverse)
#> — Attaching core tidyverse packages — tidyverse 2.0.0
#> ✓ dplyr      1.1.4      ✓ readr      2.1.5
#> ✓ forcats    1.0.0      ✓ stringr    1.5.1
#> ✓ ggplot2    3.5.1      ✓ tibble     3.2.1
#> ✓ lubridate  1.9.3      ✓ tidyr      1.3.1
#> ✓ purrr      1.0.2
#> — Conflicts — tidyverse_conflicts()
#> × dplyr::filter() masks stats::filter()
#> × dplyr::lag()    masks stats::lag()
#> i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

Set OAuth2 details from Schwab API Documentation

charles SCHWAB

Developer Portal

HomeAPI ProductsUser Guides

AppsCreate Company

App Details - [REDACTED]

App Details

App Analytics

App Machine Name:

[REDACTED]

Creator:

[REDACTED]

Create Date:

06/14/2024

App Description:

Just an app created so that I can experiment using the Schwab Develop

Environment

Production

Status

Ready For Use

Callback URL

https://127.0.0.1

Order Limit

120

Key Issued Date

06/14/2024

Last Modified Date

06/21/2024

APIs

Accounts and Trading Production

Market Data Production

App Key

👁

Secret

👁

Project Specific Data Notes:

- Environment is "Production"
- Status is "Ready for Use"
- APIs:
 - Accounts and Trading Production
 - Market Data Production

```
# 'App Key' from the App Details screen:
client_id <- "ENCRYPTEDSTRING1"

# 'Secret' from App Details screen:
client_secret <- "ENCRYPTEDSTRING2"

auth_uri <- "https://api.schwabapi.com/v1/oauth/authorize"
token_uri <- "https://api.schwabapi.com/v1/oauth/token"

# Elsewhere in OAuth 2.0 reading, I thought 'scope' was a required field when
# requesting access, but it seems Schwab API provides this as a response when
# the access token is issued. So... not necessary (?):
scope <- "" # Does not seem to be used in Schwab API documentation (?)

# AKA: "Callback URL" per Schwab API documentation:
redirect_uri <- "https://127.0.0.1"
```

Step 1: App Authorization


Developer Portal

[Home](#)
[API Products](#)
[User Guides](#)

Step 1: App Authorization

This endpoint authorizes a specific App to access to Protected Resources on behalf of the Resource Owner (User).

An Application passes the parameters of a registered App to direct the Flow to LMS. Once CAG activities are completed in LMS, an Authorization Code ("code") is returned in the landing URL following a redirect. The "code" parameter will be used in Step 2 to create the initial set of Refresh and Access Tokens. The Callback URL will be the "host" of this Application's landing page.

Following the CAG activities:

- An Authorization Code will be provided and can be exchanged for an Access Token in the next step.
- This Access Token can be used to call API Product endpoints for Protected Resource data after rest of this Flow is completed. An Access Token is valid for 30 minutes on the Trader API.
- Once a Refresh Token is invalidated or expired, the CAG activities will need to be completed again to restart the OAuth flow. A Refresh Token is valid for 7 days.

Request Template - Authorization URL

`https://api.schwabapi.com/v1/oauth/authorize?client_id={CONSUMER_KEY}&redirect_uri={APP_CALLBACK_URL}`

Response Template - Final landing URL

`https://{APP_CALLBACK_URL}/?code={AUTHORIZATION_CODE_GENERATED}&session={SESSION_ID}`

The website will be redirected to a 404 page, but the address bar will contain "code" needed for the next step.

Step 2: Access Token Creation

Based on the above, I build the "Authorization URL":



```
auth_url <- paste0(
  auth_uri,
  "?",
  "client_id=",
  client_id,
  "&redirect_uri=",
  redirect_uri
)

# The resulting URL gets pasted into a web browser
auth_url
#> [1] "https://api.schwabapi.com/v1/oauth/authorize?
client_id=ENCRYPTEDSTRING1&redirect_uri=https://127.0.0.1"
```

I paste the above `auth_url` in a web browser, which takes me to the “login micro site” (LMS) and enter my login and password (for Schwab customer account – **NOT** the developer site!), and then approving which sub-accounts to include, a `returned_auth_url` is displayed in the browser, which contains an `auth_code` embedded in it.



```
returned_auth_url <- c(
  "https://127.0.0.1/?
  code=C0.ENCRYPTEDSTRING3.ENCRYPTEDSTRING4%40&session=7ce316f9-b114-4cbd-88fa-
  422ec0fc84c6"
)
```

Function to extract that `auth_code` from `returned_auth_url` .

NOTE: The “code” within this request must be URL decoded prior to making the request. For example, this should end in ‘@’ instead of ‘%40’ when used in the request.



```
get_auth_code <- function(URL) {
  return(paste0(str_sub(str_extract(URL, ".*%"), start = 2L, end = -2L), "@"))
}

auth_code <- get_auth_code(returned_auth_url[length(returned_auth_url)])

auth_code
#> [1] "C0.ENCRYPTEDSTRING3.ENCRYPTEDSTRING4@"
```

Step 2: Access Token Creation

Developer Portal

[Home](#) [API Products](#) [User Guides](#)

Step 2: Access Token Creation

POST <https://api.schwabapi.com/v1/oauth/token>

This first POST call to the `/oauth/token` endpoint exchanges the “code” (authorization_code), returned above, for the initial “access_token”. This is used to access Protected Resources from an API Product. An Access Token is valid for 30 minutes on the Trader API.

The “code” within this request must be URL decoded prior to making the request. For example, this should end in ‘@’ instead of ‘%40’ when used in the request.

“Access Token” - Request Example (cURL)

```
curl -X POST https://api.schwabapi.com/v1/oauth/token \
-H 'Authorization: Basic {BASE64_ENCODED_Client_ID:Client_Secret}' \
-H 'Content-Type: application/x-www-form-urlencoded' \
-d 'grant_type=authorization_code&code={AUTHORIZATION_CODE_VALUE}&redirect_uri=https://example_url.com/callback_example'
```

Response Example (body)

Example - Access Token Response

```
{
  "expires_in": 1800, //Number of seconds access_token is valid for
  "token_type": "Bearer",
  "scope": "api",
  "refresh_token": "{REFRESH_TOKEN_HERE}", //Valid for 7 days
  "access_token": "{ACCESS_TOKEN_HERE}", //Valid for 30 minutes
  "id_token": "{JWT_HERE}"
}
```

A Trader API access token is valid for 30 minutes. A Trader API refresh token is valid for 7 days.

Step 3: Make an API Call

```
encoded_credentials <- base64_encode(  
  charToRaw(  
    paste(  
      client_id,  
      client_secret,  
      sep = ":"  
    )  
  )  
)  
  
encoded_credentials  
#> [1] "ENCRYPTEDSTRING5=="  
  
headers <- c(  
  "Authorization" = paste("Basic", encoded_credentials),  
  "Content-Type" = "application/x-www-form-urlencoded"  
)  
  
body <- c(  
  "grant_type" = "authorization_code",  
  "code" = auth_code,  
  "redirect_uri" = redirect_uri  
)  
  
req_access_token <- request(base_url = token_uri) %>%  
  req_headers(!!!headers) %>%  
  req_body_form(!!!body) %>%  
  print()  
#> <httr2_request>  
#> POST https://api.schwabapi.com/v1/oauth/token  
#> Headers:  
#> • Authorization: 'Basic  
#> ENCRYPTEDSTRING5=='  
#> • Content-Type: 'application/x-www-form-urlencoded'  
#> Body: form encoded data
```

Now that the request for the access token is created, processing the request fails as seen below, and I cannot see why it is coming back as a “Bad authorization code” as seen in the verbose output. I have diligently tried this at least a half dozen times within the 30 minute time frame with similar results each time.

```
resp_access_token <- req_perform(req_access_token, verbosity = 3)  
#> * Host api.schwabapi.com:443 was resolved.  
#> * IPv6: (none)  
#> * IPv4: 23.43.85.40, 23.43.85.4  
#> * Trying 23.43.85.40:443...  
#> * schannel: disabled automatic use of client certificate  
#> * Connected to api.schwabapi.com (23.43.85.40) port 443  
#> * using HTTP/1.x  
#> -> POST /v1/oauth/token HTTP/1.1  
#> -> Host: api.schwabapi.com  
#> -> User-Agent: httr2/1.0.6 r-curl/6.0.1 libcurl/8.10.1  
#> -> Accept: */*  
#> -> Accept-Encoding: deflate, gzip  
#> -> Authorization: Basic ENCRYPTEDSTRING5==
```

```

#> -> Content-Type: application/x-www-form-urlencoded
#> -> Content-Length: 150
#> ->
#> >>
grant_type=authorization_code&code=C0.ENCRYPTEDSTRING3.ENCRYPTEDSTRING4%40&redirect_
#> * upload completely sent off: 150 bytes
#> <- HTTP/1.1 400 Bad Request
#> <- Content-Type: application/json
#> <- Schwab-Client-Correlid: 946cb6f0-edbf-fa1e-4812-ENCRYPTEDSTRING6
#> <- F5IP: 162.93.231.14
#> <- Cache-Control: no-store
#> <- Pragma: no-cache
#> <- Expires: -1
#> <- Access-Control-Allow-Origin:
#> <- Access-Control-Allow-Headers: *, X-Authorization
#> <- Access-Control-Max-Age: 3628800
#> <- Access-Control-Allow-Methods: GET, POST, OPTIONS
#> <- Date: Wed, 12 Feb 2025 00:15:50 GMT
#> <- Connection: close
#> <- Strict-Transport-Security: max-age=30
#> <-
#> << {"error":"unsupported_token_type","error_description":"400 Bad Request: \"
{\\\"error_description\\\":\\\"Bad authorization code: AuthorizationCode has
expired,expiration=1739299531260,now=1739319350382
\\\",\\\"error\\\":\\\"invalid_request\\\"}\\\""}
#> * schannel: server closed abruptly (missing close_notify)
#> * closing connection #0
#> Error in `req_perform()`:
#> ! Failed to perform HTTP request.
#> Caused by error in `curl::curl_fetch_memory()`:
#> ! Failure when receiving data from the peer [api.schwabapi.com]

```

Standard output and standard error



```
-- nothing to show --
```

Session info



```

sessioninfo::session_info()
#> - Session info -----
#> setting value
#> version R version 4.4.2 (2024-10-31 ucrt)
#> os Windows 11 x64 (build 22621)
#> system x86_64, mingw32
#> ui RTerm
#> language (EN)
#> collate English_United States.utf8
#> ctype English_United States.utf8
#> tz America/New_York
#> date 2025-02-11
#> pandoc 3.2 @ C:/Program Files/RStudio/resources/app/bin/quarto/bin/tools/
(via rmarkdown)
#>
#> - Packages -----
#> package * version date (UTC) lib source
#> askpass 1.2.1 2024-10-04 [1] CRAN (R 4.4.2)

```

```

#> cli 3.6.3 2024-06-21 [1] CRAN (R 4.4.2)
#> colorspace 2.1-1 2024-07-26 [1] CRAN (R 4.4.2)
#> curl 6.0.1 2024-11-14 [1] CRAN (R 4.4.2)
#> digest 0.6.37 2024-08-19 [1] CRAN (R 4.4.2)
#> dplyr * 1.1.4 2023-11-17 [1] CRAN (R 4.4.0)
#> evaluate 1.0.1 2024-10-10 [1] CRAN (R 4.4.2)
#> fansi 1.0.6 2023-12-08 [1] CRAN (R 4.4.0)
#> fastmap 1.2.0 2024-05-15 [1] CRAN (R 4.4.0)
#> forcats * 1.0.0 2023-01-29 [1] CRAN (R 4.4.0)
#> fs 1.6.5 2024-10-30 [1] CRAN (R 4.4.2)
#> generics 0.1.3 2022-07-05 [1] CRAN (R 4.4.0)
#> ggplot2 * 3.5.1 2024-04-23 [1] CRAN (R 4.4.0)
#> glue 1.8.0 2024-09-30 [1] CRAN (R 4.4.2)
#> gtable 0.3.6 2024-10-25 [1] CRAN (R 4.4.2)
#> hms 1.1.3 2023-03-21 [1] CRAN (R 4.4.0)
#> htmltools 0.5.8.1 2024-04-04 [1] CRAN (R 4.4.0)
#> http2 * 1.0.6 2024-11-04 [1] CRAN (R 4.4.2)
#> knitr 1.49 2024-11-08 [1] CRAN (R 4.4.2)
#> lifecycle 1.0.4 2023-11-07 [1] CRAN (R 4.4.0)
#> lubridate * 1.9.3 2023-09-27 [1] CRAN (R 4.4.0)
#> magrittr 2.0.3 2022-03-30 [1] CRAN (R 4.4.0)
#> munsell 0.5.1 2024-04-01 [1] CRAN (R 4.4.0)
#> openssl * 2.2.2 2024-09-20 [1] CRAN (R 4.4.2)
#> pillar 1.9.0 2023-03-22 [1] CRAN (R 4.4.0)
#> pkgconfig 2.0.3 2019-09-22 [1] CRAN (R 4.4.0)
#> purrr * 1.0.2 2023-08-10 [1] CRAN (R 4.4.0)
#> R.cache 0.16.0 2022-07-21 [1] CRAN (R 4.4.2)
#> R.methodsS3 1.8.2 2022-06-13 [1] CRAN (R 4.4.0)
#> R.oo 1.27.0 2024-11-01 [1] CRAN (R 4.4.1)
#> R.utils 2.12.3 2023-11-18 [1] CRAN (R 4.4.2)
#> R6 2.5.1 2021-08-19 [1] CRAN (R 4.4.0)
#> rappdirs 0.3.3 2021-01-31 [1] CRAN (R 4.4.0)
#> readr * 2.1.5 2024-01-10 [1] CRAN (R 4.4.0)
#> reprex 2.1.1 2024-07-06 [1] CRAN (R 4.4.2)
#> rlang 1.1.4 2024-06-04 [1] CRAN (R 4.4.2)
#> rmarkdown 2.29 2024-11-04 [1] CRAN (R 4.4.2)
#> rstudioapi 0.17.1 2024-10-22 [1] CRAN (R 4.4.2)
#> scales 1.3.0 2023-11-28 [1] CRAN (R 4.4.0)
#> sessioninfo 1.2.2 2021-12-06 [1] CRAN (R 4.4.0)
#> stringi 1.8.4 2024-05-06 [1] CRAN (R 4.4.0)
#> stringr * 1.5.1 2023-11-14 [1] CRAN (R 4.4.0)
#> styler 1.10.3 2024-04-07 [1] CRAN (R 4.4.2)
#> tibble * 3.2.1 2023-03-20 [1] CRAN (R 4.4.0)
#> tidyr * 1.3.1 2024-01-24 [1] CRAN (R 4.4.0)
#> tidyselect 1.2.1 2024-03-11 [1] CRAN (R 4.4.0)
#> tidyverse * 2.0.0 2023-02-22 [1] CRAN (R 4.4.0)
#> timechange 0.3.0 2024-01-18 [1] CRAN (R 4.4.0)
#> tzdb 0.4.0 2023-05-12 [1] CRAN (R 4.4.0)
#> utf8 1.2.4 2023-10-22 [1] CRAN (R 4.4.0)
#> vctrs 0.6.5 2023-12-01 [1] CRAN (R 4.4.0)
#> withr 3.0.2 2024-10-28 [1] CRAN (R 4.4.2)
#> xfun 0.49 2024-10-31 [1] CRAN (R 4.4.2)
#> yaml 2.3.10 2024-07-26 [1] CRAN (R 4.4.1)
#>
#> [1] C:/Users/KQV2764/AppData/Local/R/win-library/4.4
#> [2] C:/Program Files/R/R-4.4.2/library
#>
#>

```

r oauth-2.0 httr2

Share Improve this question Follow

asked Feb 12, 2025 at 0:50



ScottyJ

1,117 14 25

Related questions

676

[How is OAuth 2 different from OAuth 1?](#)

852

[Why Does OAuth v2 Have Both Access and Refresh Tokens?](#)

1

[Microsoft identity platform and OAuth 2.0 authorization code flow \(PKCE\) - Error "AADSTS700025"](#)

✓ Load 7 more related questions

Start asking to get answers

Find the answer to your question by asking.

[Ask question](#)

Explore related questions

r

oauth-2.0

httr2

See similar questions with these tags.